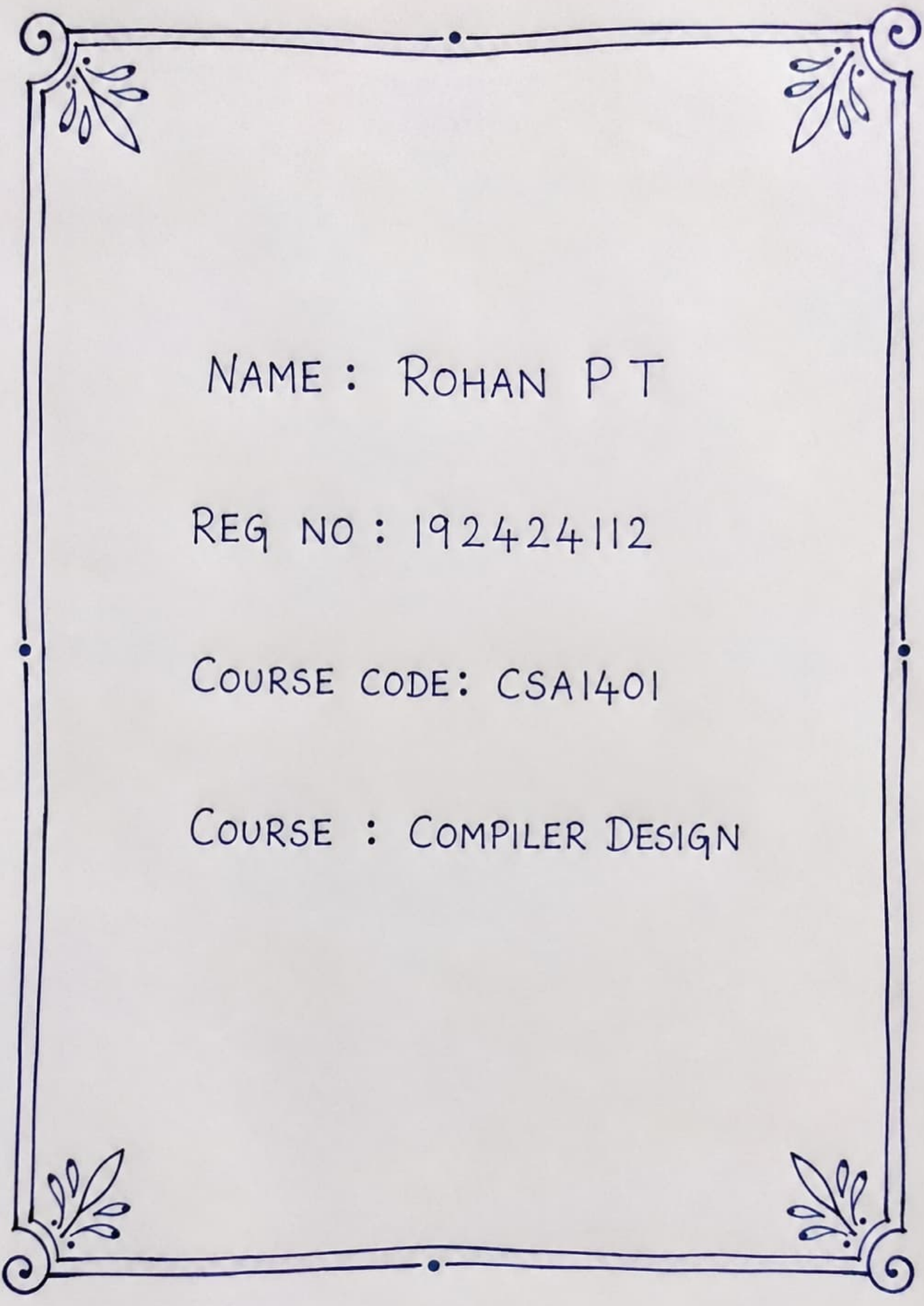




NAME : ROHAN P T

REG NO : 192424112

COURSE CODE: CSA1401

COURSE : COMPILER DESIGN

Q14 Error Analysis Task:-

a & b> Error Identification and classification:-

Line	Error	Phase
10	missing semicolon after initialization	Syntax
12	SumArray called with 1 argument;	Semantic
14	missing semicolon after printf(...)	Syntax
4	loop condition $i \leq 11; = n$	logical
14	division by zero (a/b where $b=0$)	Runtime
14	dereferencing uninitialized pointer	Runtime

Note:

NO lexical errors are Present - all tokens are well-formed
the issue are structural / meaning level

c> Compilation behaviour:-

- * Detected at compile time: the two missing semicolon (line 10, 14) - the parser cannot build a valid syntax tree without them, so compilation halts here first
- * Detected at compile time (semantic): the argument-count mismatch on line 12 - flagged if a prototype is visible; old-style c may only warn

* only appear at runtime: division by zero (line 17) and the invalid pointer dereference (line 19) - these are syntactically and semantically valid statements.

* logical error (line 4): never "detected" by the compiler - the code is just reads one element beyond the array.

* why early error blocks later detection: A compiler proceeds sequentially - lexical $\rightarrow$ syntax $\rightarrow$ semantic $\rightarrow$ code generation. if the parser cannot build a complete parse tree (e.g. missing semicolon).

d) Corrected Program:-

```
1 #include <stdio.h>
2 int sumArray (int arr[], int n) {
3     int i, sum = 0;
4     for (i=0; i<n; i++) {
5         sum = sum + arr[i];
6     }
7     return sum;
8 }
9 int main () {
10     int arr[5] = {1, 2, 3, 4, 5};
11     int total = sumArray(arr, 5);
```

```
12 if (total == 15) {
13     printf("Correct");
14 }
15 int a=10, b=a;
16 int c = a/b;
17 int p_val = 20;
18 int *p = &p_val;
19 *p = 20;
20 return 0;
21 }
```

e) Impact Analysis (these errors):-

1. Division by zero (line 17) : causes an immediate crash (SIGFPE on Linux / Unix) since integer division by zero is undefined behaviour in C.
2. wild pointer difference (line 19)
3. off-by-one loop bound (line 4): reading `arr[n]` beyond valid indices.

f) Advanced Compiler Perspective:-

- * Static Analysis integration : flag likely - null / uninitialized pointers and possible division - by - zero pattern at compile time using data - flow analysis (e.g. Gcc - fuzzer)
- * Bounds - checking instrumentation : insert runtime array bounds check (as in Rust, or via AddressSanitizer) that trap on out - of - range access instead of silent corruption
- * Better Diagnostics : point to the exact expected token with a caret under the following column instead of a generic syntax error message.

Qn ② Lexical Errors:-

a) Error Identification:-

line 3 - 1num - identifier rule - cannot start with a digit

line 4 - rate @ - symbol rule - '@' not a legal identifier.

line 5 → 'abc' → char literal rule - must hold exactly one char

line 6 → 50 \$ 20 → symbol rule

line 7 → num # 1 → symbol rule

line 8 → % count → symbol rule

line 9 → unterminated string → string literal rule

line 10 → 09 → constant formation

Note:

line 15 (int_valid = 10); is actually valid - identifiers may start with an underscore. worth mentioning as a contrast case.

b) Error Classification:-

* Identifier Rule

Example:

→ identifier starts with digit

→ contains illegal characters

→ contains '-'

* Constant Rule

Example:

→ malformed decimal

→ invalid hexadecimal

→ invalid octal

* Character Literal

Eg:
m

→ Multiple character → missing quote

* String Literal

Eg:

→ missing closing quotation

* Illegal Symbols

Eg: @, #, \$

c) Taken Correction:-

Convert each invalid token into a legal one

Example format:

invalid
m
value

correct
m
value2

d) Compiler Behaviour Analysis:-

A lexical analyser reads source character and groups them into the longest valid token it can match

against defined token patterns. when a sequence matches no valid pattern (e.g. '@' or '\$') , it reports a lexical error.

- Continue Scanning
- skip invalid character
- report multiple errors together

e) Advanced thinking:-

- * State s_0 (start) $\rightarrow$ on $[a-zA-Z] \rightarrow$ go to s_1 (accepting)
- * State $s_0 \rightarrow$ on $[0-9] \rightarrow$ go to error state (reject)
- * State $s_1 \rightarrow$ on $[a-zA-Z0-9] \rightarrow$ stay in s_1 (accepting)
- * State $s_1 \rightarrow$ on any other character $\rightarrow$ token ends, emit identifier

Q3> Multi Phase Error:-

a) identify error:-

line	Error	Description
3	return a/b	missing semicolon (;)
7	int # value = 20;	invalid identifier
8	float num = 15.5	missing semicolon
10	result = divide(10);	Function divide() requires argument
11	if (result = 5)	assignment operator (=) used instead of (==)

b) Classify by Compiler Phase:-

I Lexical error

* line 7: int # value = 20;

* invalid symbol # in identifier

II Syntax Error:

* line 3: missing ;

* line 8: missing ;

* line 12: missing ;

III Semantic error:

* line 10: incorrect number of arguments in function call

* line 11: Assignment (=) instead of (==)

IV Runtime error:

* line 16: Division by zero

* line 18: Array out-of-bound access

c) Compilation vs Execution:-

I Error detection at compile time

II Error detected only during execution

- division by zero

- Array index out of bound

- infinite loop

d) correct code:-

```
#include <stdio.h>
```

```
int divide (int a, int b) {
```

```
    if (b == 0) {
```

```
        printf ("Division by zero is not allowed");
```

```
        return 0;
```

```
    }
```

```
    return a/b;
```

```
}
```

```
int main () {
```

```
    int value = 80;
```

```
    float num = 15.5;
```

```
    int result;
```

```
    result = divide (10, 2);
```

```
    if (result == 5) {
```

```
        printf ("result is 5\n");
```

```
    }
```

```
    int x = 10;
```

```
    int y = 2;
```

```
    int z = x/y;
```

```
    printf ("x = %.d\n", x);
```

```
    int arr [4] = {1, 2, 3, 4};
```

```
    printf (arr [3]);
```

```
    int temp = 25;
```

```
    int *ptr = &temp;
```

```
    int total;
```

```
    total = x + y + result;
```

```
    return 0;
```

```
}
```